

Go for Senior .NET Engineers

Phase 5: Toolchain & Build System

6 Lessons: Modules, Build, Cross-Compile, Linting, Workspaces, Docker

Current as of Go 1.25.6 (January 2026) — includes tool directives, go.work, build tags

Lesson 43: Go Modules — go.mod, go.sum, Versioning

Go Modules is Go's dependency management system, analogous to NuGet in .NET. Every Go project starts with a go.mod file that declares the module path and its dependencies. There is no packages.config, no .csproj — just go.mod.

43.1 — Creating a Module

```
# Initialize a new module
mkdir myservice && cd myservice
go mod init github.com/yourorg/myservice

# This creates go.mod:
module github.com/yourorg/myservice

go 1.25
```

43.2 — NuGet vs Go Modules

C# / .NET

```
// C# - NuGet
// .csproj:
<PackageReference
  Include="Newtonsoft.Json"
  Version="13.0.3" />

// Install:
dotnet add package Newtonsoft.Json

// Restore:
dotnet restore

// Source: nuget.org
```

Go

```
// Go - Modules
// go.mod:
require (
  github.com/gin-gonic/gin v1.9.1
)

// Install:
go get github.com/gin-gonic/gin@v1.9.1

// Tidy (remove unused):
go mod tidy

// Source: any Git repo (proxy.golang.org)
```

43.3 — Essential Module Commands

Command	Purpose
go mod init <path>	Create go.mod for new module
go get <pkg>@<ver>	Add or update a dependency
go mod tidy	Sync go.mod with actual imports
go mod download	Pre-download all deps (like dotnet restore)
go mod verify	Verify checksums against go.sum

<code>go mod vendor</code>	Copy deps into vendor/ directory
<code>go mod graph</code>	Print dependency graph
<code>go mod why <pkg></code>	Explain why a dep is needed

43.4 — go.sum — The Lock File

```
# go.sum contains cryptographic hashes of every dependency
# Similar to packages.lock.json in .NET
# ALWAYS commit go.sum to version control

# go.sum entry format:
# module version hash
github.com/gin-gonic/gin v1.9.1 h1:4+fr/el...
github.com/gin-gonic/gin v1.9.1/go.mod h1:bC...

# Verified against Go Checksum Database (sum.golang.org)
# Prevents supply-chain attacks by ensuring reproducible builds
```

43.5 — Semantic Import Versioning

```
// Major version 0 or 1: normal import
import "github.com/lib/pq"

// Major version 2+: version in import path
import "github.com/go-redis/redis/v9"

// This is unique to Go – different major versions
// are different packages that can coexist
// (unlike NuGet where you can only have one version)
```

TIP: go mod tidy is your best friend. Run it after adding/removing imports. It adds missing deps and removes unused ones. It is the Go equivalent of running 'dotnet restore' + cleaning up unused packages.

Lesson 44: go build — Compilation & Cross-Compilation

Go compiles to a single static binary with zero runtime dependencies. No .NET runtime to install, no JVM, no Docker base image with a framework. This is one of Go's biggest operational advantages for deployment.

44.1 — Basic Build

```
# Build the current module
go build -o myservice ./cmd/myservice

# Build and run in one step
go run ./cmd/myservice

# Install binary to $GOPATH/bin (or $GOBIN)
go install ./cmd/myservice

# Build flags
go build -v           # verbose (print package names)
go build -race        # enable race detector (testing only)
go build -trimpath     # remove local paths from binary
```

44.2 — Cross-Compilation (Go's Superpower)

```
# Build for Linux from macOS or Windows – one line!
GOOS=linux GOARCH=amd64 go build -o myservice

# Build for Windows from Linux
GOOS=windows GOARCH=amd64 go build -o myservice.exe

# Build for ARM (Raspberry Pi, AWS Graviton)
GOOS=linux GOARCH=arm64 go build -o myservice

# No SDK install, no special toolchain, no Docker buildx
# Just set two environment variables and build
```

44.3 — dotnet publish vs go build

C#/.NET

```
// C# – dotnet publish
// Self-contained (large binary):
dotnet publish -c Release \
  -r linux-x64 \
  --self-contained true \
  -p:PublishSingleFile=true

// Output: ~70-150 MB
// Requires .NET runtime or self-contained
```

Go

```
// Go – go build
// Always self-contained:
GOOS=linux GOARCH=amd64 \
  go build -o myservice
```

```
// Cross-compile needs target RID
```

```
// Output: ~5-20 MB  
// Zero dependencies, static binary  
// Cross-compile: just GOOS + GOARCH
```

44.4 — Build Tags (Conditional Compilation)

```
// Build tags = C# #if directives  
  
//go:build linux  
// +build linux  
package platform  
  
func GetConfigPath() string {  
    return "/etc/myservice/config.yaml"  
}  
  
// File: config_windows.go  
//go:build windows  
package platform  
  
func GetConfigPath() string {  
    return `C:\ProgramData\myservice\config.yaml`  
}  
  
// Custom build tags  
//go:build integration  
// Run: go test -tags=integration ./...
```

44.5 — Embedding Version Info with ldflags

```
// main.go  
var version = "dev" // overridden at build time  
  
// Build with version injected:  
// go build -ldflags "-X main.version=v1.2.3" -o myservice  
  
// In Makefile or CI:  
// VERSION=$(git describe --tags --always)  
// go build -ldflags "-X main.version=${VERSION}" -o myservice
```

TIP: Go binaries are statically linked by default. They contain everything needed to run. A typical Go microservice binary is 5-20 MB. Compare that to a self-contained .NET publish which is 70-150 MB. This makes Go ideal for minimal Docker images (FROM scratch).

Lesson 45: Go Toolchain — go vet, go fmt, go generate

Go ships with a comprehensive built-in toolchain. Unlike .NET where you need Roslyn analyzers, StyleCop, and code generators as separate NuGet packages, Go includes formatting, static analysis, and code generation out of the box.

45.1 — go fmt / gofmt (Mandatory Formatting)

```
# Format all files in the module
gofmt -w .

# Or use go fmt (runs gofmt on packages)
go fmt ./...

# goimports = gofmt + auto-manage imports
# Install: go install golang.org/x/tools/cmd/goimports@latest
goimports -w .

# There is NO formatting debate in Go.
# gofmt is the standard. Everyone uses it.
# Tabs, not spaces. Brace on same line. Period.
```

45.2 — go vet (Static Analysis)

```
# Run static analysis on all packages
go vet ./...

# Catches common bugs:
# - Printf format string mismatches
# - Unreachable code
# - Incorrect struct tags
# - Suspicious loop variable capture
# - Copying sync.Mutex
# - Nil pointer potential dereferences

# Always run go vet in CI alongside go test
```

45.3 — go generate (Code Generation)

```
// Add a generate directive in your Go source:
//go:generate stringer -type=Status
//go:generate mockgen -source=repo.go -destination=mock_repo.go

// Run all generators in the module:
// go generate ./...

// Common generators:
// stringer    - Generate String() for enums
```

```
// mockgen      - Generate mock implementations
// protoc-gen-go - Generate gRPC code from .proto
// sqlc         - Generate type-safe SQL code
// ent          - Generate ORM code from schema
```

45.4 — Tool Directives in go.mod (Go 1.24+)

```
// go.mod can now declare tool dependencies:
module github.com/yourorg/myservice

go 1.25

tool (
    golang.org/x/tools/cmd/stringer
    go.uber.org/mock/mockgen
    github.com/sqlc-dev/sqlc/cmd/sqlc
)

// Install all tools: go install tool
// Run: go tool stringer -type=Status
// No more Makefile hacks for tool versioning!
```

TIP: Go 1.24 added the 'tool' directive in go.mod. This is a game-changer: you can now version your code generation tools alongside your module, ensuring every developer and CI system uses the same tool versions. Similar to .NET's dotnet-tools.json.

Lesson 46: Linting — golangci-lint & Static Analysis

golangci-lint is the de facto standard linter aggregator for Go. It runs 100+ linters in parallel and is the Go equivalent of having Roslyn analyzers, StyleCop, SonarQube, and ReSharper inspections combined into one fast tool.

46.1 — Installing & Running

```
# Install (recommended: binary)
curl -sSfL https://raw.githubusercontent.com/golangci-lint/golangci-lint/HEAD/install.sh | sh

# Or via go install
go install github.com/golangci-lint/golangci-lint/cmd/golangci-lint@latest

# Run on entire project
golangci-lint run ./...

# Run with auto-fix
golangci-lint run --fix ./...
```

46.2 — Configuration (.golangci.yml)

```
# .golangci.yml (project root)
run:
  timeout: 5m

linters:
  enable:
    - errcheck      # unchecked errors
    - govet         # go vet checks
    - staticcheck   # comprehensive static analysis
    - unused        # unused code
    - gosimple      # simplify code
    - ineffassign   # unused assignments
    - gocritic      # opinionated style
    - revive        # extensible linter (replaces golint)
    - exhaustive    # exhaustive enum switches
    - nilnil        # returning nil, nil
    - prealloc      # suggest slice pre-allocation

linters-settings:
  errcheck:
    check-type-assertions: true
  gocritic:
    enabled-tags:
      - diagnostic
      - style
      - performance
```


46.3 — Key Linters for C# Developers

Linters	What It Catches (C# Equivalent)
errcheck	Unchecked errors (like ignoring Task results)
staticcheck	Comprehensive checks (like Roslyn analyzers)
govet	Suspicious constructs (built-in)
gosec	Security issues (like SonarQube)
ineffassign	Unused variable assignments
exhaustive	Non-exhaustive type switches (like CS8509)
gocritic	Style & performance (like ReSharper hints)
revive	Extensible linting rules (like StyleCop)

46.4 — CI Integration

```
# GitHub Actions example:
- name: golangci-lint
  uses: golangci/golangci-lint-action@v6
  with:
    version: latest

# Typical CI pipeline for Go:
# 1. go build ./...
# 2. go test -race ./...
# 3. go vet ./...
# 4. golangci-lint run ./...
```

WARNING: Do NOT skip errcheck in your linter config. Unchecked errors are the Go equivalent of swallowing exceptions in C#. The errcheck linter catches every place where you ignore a returned error.

Lesson 47: Workspaces — go.work & Multi-Module Development

Go Workspaces (Go 1.18+) solve the multi-module development problem. When you have a monorepo with multiple services sharing a common library, go.work lets you develop them together without publishing intermediate versions — similar to .NET solution files (.sln) grouping multiple projects.

47.1 — The Problem Workspaces Solve

C# / .NET

```
// C# - .sln multi-project
MyApp.sln
├─ Api.csproj (references Shared)
├─ Worker.csproj (references Shared)
└─ Shared.csproj

// ProjectReference:
<ProjectReference Include=
  "..\Shared\Shared.csproj" />

// Changes in Shared are instantly
// visible to Api and Worker
```

Go

```
// Go - go.work
go.work
├─ api/go.mod
├─ worker/go.mod
└─ shared/go.mod

// go.work file:
go 1.25
use (
  ./api
  ./worker
  ./shared
)
// Changes in shared are instantly visible
```

47.2 — Setting Up a Workspace

```
# Project structure:
# myproject/
#   api/go.mod      (module github.com/org/myproject/api)
#   worker/go.mod   (module github.com/org/myproject/worker)
#   shared/go.mod   (module github.com/org/myproject/shared)

# Initialize workspace from project root:
cd myproject
go work init ./api ./worker ./shared

# This creates go.work:
go 1.25

use (
  ./api
  ./worker
  ./shared
)

# Now: go build, go test, go run all work across modules
# shared changes are reflected instantly – no go get needed
```

47.3 — Workspace Commands

Command	Purpose
<code>go work init [dirs...]</code>	Create <code>go.work</code> with initial modules
<code>go work use [dir]</code>	Add a module to the workspace
<code>go work use -r .</code>	Recursively find and add all <code>go.mod</code> files
<code>go work sync</code>	Sync dependency versions across modules
<code>go work edit</code>	Programmatically edit <code>go.work</code>

47.4 — Workspace Best Practices

```
# 1. DO NOT commit go.work to version control
# (CI builds each module independently via go.mod)
echo "go.work" >> .gitignore
echo "go.work.sum" >> .gitignore

# 2. Each module's go.mod should work independently
# go.work is a developer convenience, not a build requirement

# 3. Use replace directives as fallback for non-workspace users
# go.mod:
# replace github.com/org/myproject/shared => ../shared
# (but prefer go.work over replace)

# 4. In CI, build each module separately:
# cd api && go build ./... && go test ./...
# cd worker && go build ./... && go test ./...
```

TIP: `go.work` is for local development only. Think of it as your personal `.sln` file. CI/CD should always build using each module's `go.mod` independently to ensure published dependency versions are correct.

Lesson 48: Docker & Deployment — Minimal Images & CI/CD

Go's static binaries make it the easiest language to containerize. A Go service can run in a FROM scratch image (literally empty — just the binary), producing Docker images as small as 5-20 MB. Compare this to .NET's 200+ MB images with the runtime.

48.1 — Multi-Stage Dockerfile

```
# Stage 1: Build
FROM golang:1.25-alpine AS builder

WORKDIR /app
COPY go.mod go.sum ./
RUN go mod download

COPY . .
RUN CGO_ENABLED=0 GOOS=linux go build \
    -trimpath -ldflags="-s -w" \
    -o /app/myservice ./cmd/myservice

# Stage 2: Run (EMPTY base image)
FROM scratch
COPY --from=builder /etc/ssl/certs/ca-certificates.crt /etc/ssl/certs/
COPY --from=builder /app/myservice /myservice

EXPOSE 8080
ENTRYPOINT ["/myservice"]

# Final image: ~10-20 MB (binary + TLS certs)
```

48.2 — .NET Dockerfile vs Go Dockerfile

C# / .NET

```
// .NET Dockerfile
FROM mcr.microsoft.com/dotnet/sdk:8.0
AS build
WORKDIR /src
COPY *.csproj .
RUN dotnet restore
COPY . .
RUN dotnet publish -c Release \
    -o /app

FROM mcr.microsoft.com/dotnet/
aspnet:8.0
COPY --from=build /app .
ENTRYPOINT ["dotnet", "MyApp.dll"]
# Image: ~200-300 MB
```

Go

```
// Go Dockerfile
FROM golang:1.25-alpine AS build

WORKDIR /app
COPY go.* ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 go build \
    -o /app/svc ./cmd/svc

FROM scratch
COPY --from=build /app/svc /svc
ENTRYPOINT ["/svc"]
# Image: ~10-20 MB
```

48.3 — Build Flags for Production

```
# Production build flags explained:
CGO_ENABLED=0 # Pure Go (no C dependencies) → static binary
-trimpath      # Remove local file paths from binary
-ldflags="-s"  # Strip symbol table (smaller binary)
-ldflags="-w"  # Strip DWARF debug info (smaller binary)

# Combined:
CGO_ENABLED=0 go build \
  -trimpath \
  -ldflags="-s -w -X main.version=v1.2.3" \
  -o myservice ./cmd/myservice

# Typical size reduction: 30-40% smaller binary
```

48.4 — Deployment Patterns

Pattern	Description
FROM scratch	Empty image, just the binary + TLS certs
FROM alpine:latest	Tiny Linux base, has shell for debugging
FROM distroless	Google's minimal images, no shell/package manager
Systemd unit	Deploy binary directly to a Linux VM
AWS Lambda	Go compiles to provided.al2023 runtime
Kubernetes	Typical target for Go microservices

48.5 — CI/CD Pipeline (GitHub Actions)

```
name: Go CI
on: [push, pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-go@v5
        with:
          go-version: "1.25"
      - run: go build ./...
      - run: go test -race -coverprofile=coverage.out ./...
      - run: go vet ./...
      - uses: golangci/golangci-lint-action@v6
```

48.6 — Phase 5 Summary: Toolchain Concept Map

C# / .NET Concept	Go Equivalent
.csproj / NuGet	go.mod / Go Modules
packages.lock.json	go.sum
dotnet restore	go mod download / go mod tidy
dotnet build	go build
dotnet publish (self-contained)	go build (always self-contained)
dotnet test	go test ./...
Cross-compile (RID)	GOOS + GOARCH env vars
.sln (multi-project)	go.work (multi-module workspace)
ProjectReference	go.work use directive
Roslyn Analyzers	go vet + staticcheck
StyleCop / ReSharper	golangci-lint
dotnet format	gofmt / goimports (mandatory)
Source generators	go generate
dotnet-tools.json	go.mod tool directive (Go 1.24+)
#if DEBUG / #if LINUX	//go:build tags
-p:Version=x.y.z	-ldflags "-X main.version=x.y.z"
200-300 MB Docker image	10-20 MB Docker image (FROM scratch)

TIP: The Go toolchain is deliberately minimal and opinionated. There is one formatter (gofmt), one build tool (go build), one test runner (go test). This eliminates the 'which tool should I use?' decision fatigue that plagues the .NET ecosystem with its many competing options.